**EXPERIMENT: Write a Prolog Program for backward Chaining. Incorporate required queries.**

**Aim**

To write a Prolog program to implement Backward Chaining and incorporate the required queries to prove a goal from the given facts and rules.

**Objectives**

1. To understand the concept of Backward Chaining.
2. To represent knowledge using facts and rules in Prolog.
3. To prove a goal by working backward from the conclusion.
4. To execute queries and verify whether the given goal is true.

**Algorithm**

1. Start the program.
2. Define the initial facts.
3. Define rules connecting conclusions with their required conditions.
4. Take a goal or query from the user.
5. Check whether the goal is a known fact.
6. If it is not a fact, find a rule whose conclusion matches the goal.
7. Check whether all conditions of that rule can be proved.
8. Continue this process recursively until the required facts are found.
9. If all required conditions are satisfied, the goal is true.
10. Otherwise, the goal is false.
11. Stop the program.

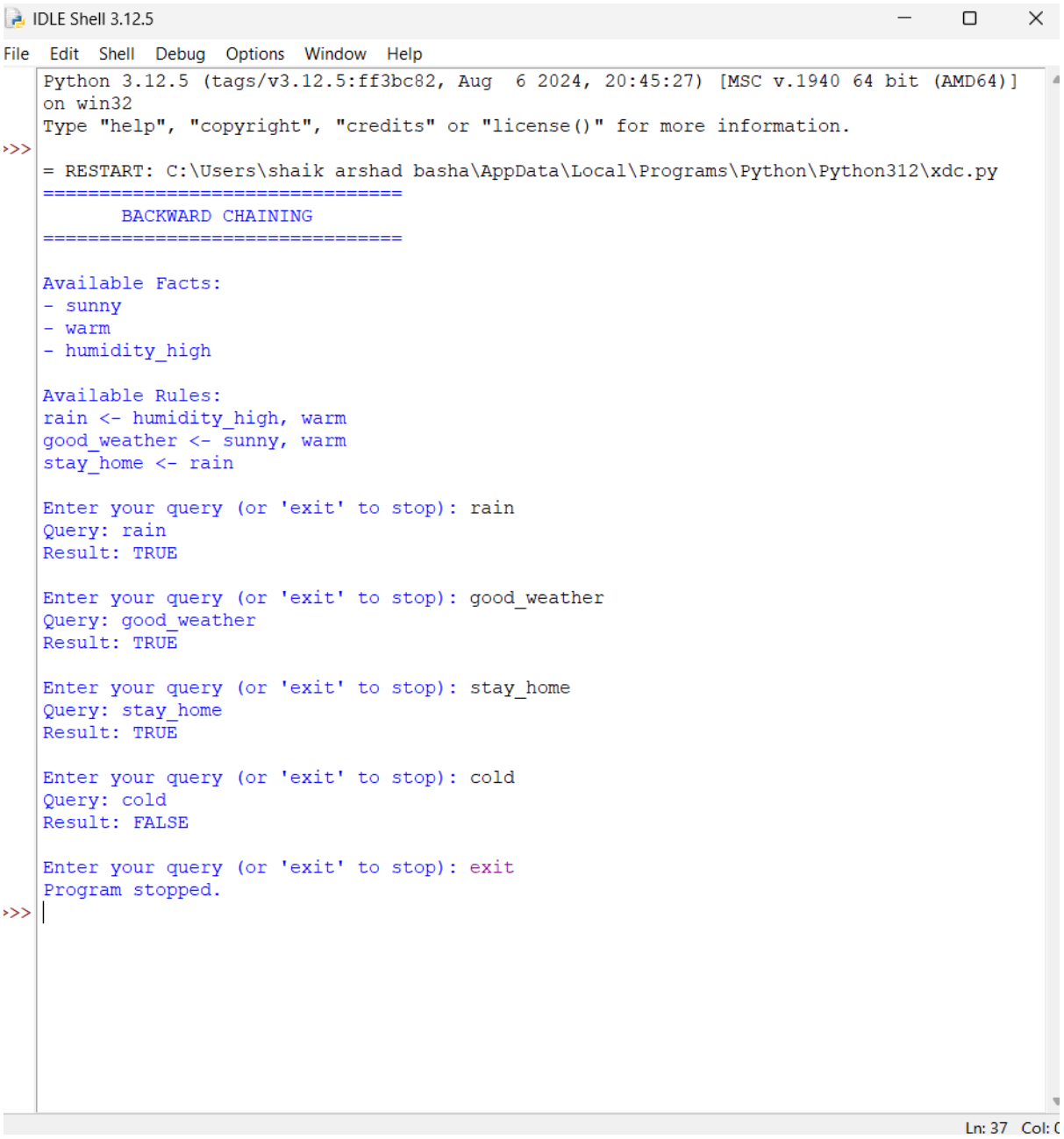
**CODE**

```
facts = {
   "sunny",
   "warm",
   "humidity_high"
}
```

```
rules = {
    "rain": ["humidity_high", "warm"],
    "good_weather": ["sunny", "warm"],
    "stay_home": ["rain"]
}
def backward_chaining(goal, visited=None):
    if visited is None:
        visited = set()
    if goal in visited:
        return False
    if goal in facts:
        return True
    visited.add(goal)
    if goal in rules:
        conditions = rules[goal]
        for condition in conditions:
            if not backward_chaining(condition, visited):
                return False
        return True
    return False
print("==================================")
print("       BACKWARD CHAINING")
print("==================================")
print("\nAvailable Facts:")
for fact in facts:
    print("-", fact)
```

```
print("\nAvailable Rules:")
for conclusion, conditions in rules.items():
    print(conclusion, "<-", ", ".join(conditions))
while True:
    query = input("\nEnter your query (or 'exit' to stop): ").lower()
    if query == "exit":
        print("Program stopped.")
        break
    result = backward_chaining(query)
    if result:
        print("Query:", query)
        print("Result: TRUE")
    else:
        print("Query:", query)
        print("Result: FALSE")
```

**OUTPUT**




## Result

The Backward Chaining algorithm was successfully implemented in Prolog. The required queries were executed, and the system successfully determined whether the given goals could be proved from the available facts and rules.

## Conclusion

Backward Chaining is a goal-driven reasoning technique. It starts with a goal and works backward through the rules to determine whether the required conditions are satisfied. Prolog naturally supports this type of reasoning through its query evaluation and rule-based inference mechanism.